WIKIPEDIA
The Free Encyclopedia

Adaptive sort

A sorting algorithm falls into the **adaptive sort** family if it takes advantage of existing order in its input. It benefits from the presortedness in the input sequence – or a limited amount of disorder for various definitions of measures of disorder – and sorts faster. Adaptive sorting is usually performed by modifying existing sorting algorithms.

Motivation

Comparison-based sorting algorithms have traditionally dealt with achieving an optimal bound of $O(n \log n)$ when dealing with time complexity. Adaptive sort takes advantage of the existing order of the input to try to achieve better times, so that the time taken by the algorithm to sort is a smoothly growing function of the size of the sequence *and* the disorder in the sequence. In other words, the more presorted the input is, the faster it should be sorted.

This is an attractive feature for a sorting algorithm because sequences that nearly sorted are common in practice. Thus, the performance of existing sorting algorithms can be improved by taking into account the existing order in the input.

Most worst-case sorting algorithms that do optimally well in the worst-case, notably heap sort and merge sort, do not take existing order within their input into account, although this deficiency is easily rectified in the case of merge sort by checking if the last element of the left-hand group is less than (or equal) to the first element of the righthand group, in which case a merge operation may be replaced by simple concatenation – a modification that is well within the scope of making an algorithm adaptive.

Examples

A classic example of an adaptive sorting algorithm is insertion sort.[1] In this sorting algorithm, the input is scanned from left to right, repeatedly finding the position of the current item, and inserting it into an array of previously sorted items.

Pseudo-code for the insertion sort algorithm follows (array X is zero-based):

```
procedure Insertion Sort (X):
    for j = 1 to length(X) - 1 do
        t ← X[j]
        i ← j
        while i > 0 and X[i - 1] > t do
            X[i] ← X[i - 1]
            i ← i - 1
        end
        X[i] ← t
    end
```

The performance of this algorithm can be described in terms of the number of inversions in the input, and then $T(n)$ will be roughly equal to $I(A) + (n - 1)$, where $I(A)$ is the number of inversions. Using this measure of presortedness – being relative to the number of inversions – insertion sort takes less time to sort the closer the array of data is to being sorted.

Other examples of adaptive sorting algorithms are adaptive heap sort, adaptive merge sort, patience sort,[2] Shellsort, smoothsort, splaysort, Timsort, and Cartesian tree sorting.[3]

See also

- Sorting algorithms

References

- Hagerup, Torben; Jyrki Katjainen (2004). *Algorithm Theory – SWAT 2004* (https://books.google.com/books?id=2E-KNhKl3gEC&q=adaptive+sort&pg=PA221). Berlin Heidelberg: Springer-Verlag. pp. 221–222. ISBN 3-540-22339-8.
- Mehta, Dinesh P.; Sartaj Sahni (2005). *Data Structures and Applications* (https://books.google.com/books?id=fQVZy1zcpJkC&q=adaptive+sort&pg=PT230). USA: Chapman & Hall/CRC. pp. 11-8–11-9. ISBN 1-58488-435-5.
- Petersson, Ola; Alistair Moffat (1992). *A framework for adaptive sorting*. Lecture Notes in Computer Science. Vol. 621. Berlin: Springer Berlin / Heidelberg. pp. 422–433. doi:10.1007/3-540-55706-7_38 (https://doi.org/10.1007%2F3-540-55706-7_38). ISBN 978-3-540-55706-7. ISSN 1611-3349 (https://search.worldcat.org/issn/1611-3349).

1. Estivill-Castro, Vladmir; Wood, Derick (December 1992). "A survey of adaptive sorting algorithms". *ACM*. **24** (4). New York, NY, USA: 441–476. CiteSeerX 10.1.1.45.8017 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.45.8017). doi:10.1145/146370.146381 (https://doi.org/10.1145%2F146370.146381). ISSN 0360-0300 (https://search.worldcat.org/issn/0360-0300). S2CID 1540978 (https://api.semanticscholar.org/CorpusID:1540978).
2. Chandramouli, Badrish; Goldstein, Jonathan (2014). *Patience is a Virtue: Revisiting Merge and Sort on Modern Processors* (https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/patsort-sigmod14.pdf) (PDF). SIGMOD/PODS.
3. Levcopoulos, Christos; Petersson, Ola (1989). "Heapsort - Adapted for Presorted Files". *WADS '89: Proceedings of the Workshop on Algorithms and Data Structures*. Lecture Notes in Computer Science. Vol. 382. London, UK: Springer-Verlag. pp. 499–509. doi:10.1007/3-540-51542-9_41 (https://doi.org/10.1007%2F3-540-51542-9_41). ISBN 978-3-540-51542-5.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Adaptive_sort&oldid=1228372133"